

Lecture 6

Integration Testing:

What it is: The second stage of testing, after individual components are tested. It focuses on finding failures detected in the component test Failure only through interaction of components. Like a component sending corrupt data to another component and causing a crash to it.

Prerequisites:

- Component tests are done.
- Any found defects are fixed.

What happens:

- Combine multiple components into bigger subsystems.
- Can be done by developers, testers, or dedicated integration teams.

Purpose:

- Check that components work correctly together.
- Detect interface or interaction failures between components.

Integration Testing- Test Basis:

Test Basis: refers to the documentation or information you use to design and perform tests.

For integration testing these test Basis could be:

Software and system design: These documents describe how the software is built and how different parts (components, modules, or subsystems) are organized.

Sequence diagrams: this is a document that describes the user journey and which components are going to be used in which sequence

Specifications of interfaces and communication protocols: Defines how components exchange data or communicate.

Use cases

Architecture at component or system level Workflows: Shows the big picture of the system, including major components and how they connect.

External interface definitions (API): Defines how your system communicates with external systems

Integration Testing - Testing object:

Test object: Test objects are the modules, components, or parts of the system that are combined and tested together to see if they work correctly as a unit. Each test object can be a small

module or a larger subsystem; integration testing gradually builds up from smaller to bigger units.

Integrate & test individual modules step by step to form larger units.

Each subsystem can be the basis for the integration of larger units.

These objects could be:

Subsystems, Databases:

Infrastructure: The underlying systems like servers, network, or cloud resources that support your software.

Example: Ensuring that multiple services can communicate correctly over a cloud network during integration.

Interfaces: Points where modules or systems connect and exchange data.

APIs, Microservices.

System & Component Integration Testing:

1. Component Integration Test

- Happens after individual components are tested.
- Focuses on how components work together.
- Often part of continuous integration.
- Usually done by **developers**.

2. System Integration Test

- Happens after or alongside system testing.
- Focuses on interaction between systems, including hardware, packages, or microservices.
- Can also test third-party interfaces.
- Usually done by **testers**.

Integration Tests Defects & failures:

Failures that we will mention can only be detected by Integration testing never by component testing. It happens through interaction of components.

Component sending corrupt data to another component and causing it to fail, (protocol failure, incompatible format)

Timing issue, data could be transferred in the correct format but at the wrong time.

The sender sent the data correctly but the receiver handled the data incorrectly.
Misinterpretation.

What are the drawbacks of doing away with integration testing, skipping the

component? finding bugs early is much cheaper and time saving. The more complex the system is the harder it is to find the place of the error. External APIs might fail silently or other components maybe corrupt and go unnoticed.

The greater the scope of an integration, the ... more difficult the isolation of defects greater the time required to fix errors

Integration Test – Test Environment:

During component testing we should reuse stubs and drivers we have already created and used during the component testing.

Integration test with drivers: import test data, log results: we should import test data through drivers to the system being tested, we should also log the results of the data that we have imported

Additional diagnostic tool (monitors) for interface monitoring; during the interaction between components data get sent a monitoring tool helps us to see some data and details being transferred between interfaces; Data being sent incorrectly, Messages arriving late or out of order, Protocol violations, Unexpected errors.

Top Down Integration Testing:

Those tests start with the components that call other components; to clarify, a component that calls 4 lower components but is not getting called is the one that is being tested first.

If we lack a higher module we provide a driver and a stub if we don't have a lower module.

Advantage: The top-level module can drive the test itself, so you save time and effort on writing extra driver code. Since you start from the top there is no need for drivers.

Disadvantages: we shall create stubs for the lower components

Bottom Up Integration Testing:

Test starts with basic components that do not call others Successive integration of components with subsequent testing.

Advantage: No placeholders required. You don't have to worry about stubs that are being called from upper modules since you now test from the lowest level.

Disadvantage: Superordinate components must be simulated by drivers. It's the opposite of top bottom you now have to worry about writing drivers

Comments: Top-down or bottom-up approach can only be used for hierarchically structured systems (not super common) In practice, a mix of both integration strategies is commonly used.

Ad-hoc Integration:

It's the idea of conducting integration tests as soon as the components are done and tested.

The goal is not full integration yet, but to see if the component can connect and interact with others without major problems.

Advantage: Time saving, as earliest possible integration

Disadvantage: Placeholders and drivers required. Since its random sometimes you will need stubs and driver both to be written.

Non-incremental Integration – big-bang Integration:

In big-bang integration, you wait until all components are fully developed. Then you integrate everything at once and test the whole system together. Sometimes, this is done even without prior component testing.

How it works:

modules (or software components) are finished. All modules are connected together in one step. The system is tested as a whole, not in smaller parts.

Disadvantages: Waiting time until the big bang is lost testing time. Failures occur all at once. Very difficult to ensure that the system works at all. Finding and correcting defects is extremely difficult

Now that we have tested all components by themselves and integrated... Why do we need to run a system test?

Because integration testing checks if components work together, but **system testing checks if the whole system works correctly and meets the requirements. Integration = “Do the parts talk to each other?” System test = “Does the system do what it’s supposed to do for the user?”

System Testing:

System test: takes place after the completion of integration test, at this point the entire system works as a one unit.

Goal: is to verify the requirements of the system.

Motivation: Earlier test stages (component and integration tests) mainly check the system from a technical and developer perspective.

System testing is to verify requirements and the whole system from a user’s perspective.

Some requirements can only be tested when the entire system is done.

It verifies that all requirements are fully and correctly implemented, not just that the software works internally.

System Test – Test Objectives:

- test the system as one entire unit (both functional and non -functional)
- Risk reduction, finding of failures (and defects)
- in case we found some defects or failures we shall not pass the system to higher testing phases.
- Create confidence in the quality of the system as a whole
- partially test the quality of data especially from a user’s perspective. Quality of data means that (data is correct, complete, consistent, valid)
- Collect information for release decision

System Test Basis:

A test basis is any document, diagram, code. or any asset that can be used to conduct and execute a test.

What can be used:

functional and non-functional requirements.

Use cases, epics, user stories.

Risk analysis report: A report that identifies potential risks in the system, such as areas where failures are most likely or would be most costly.

System Behavior Models: diagrams, flow charts that describes how the system should behave

Documentation and User Guides

System Test – Test Objects:

Test object is a part of the system that a test will be conducted upon.

Systems under test (SUT): the entire complete system that we will be testing, e.g; an entire online banking app. Even including the hardware.

Applications: some systems are made out of many different systems smaller ones, we test these smaller systems, not as how they interact internally but how they behave from a user's perspective.

Operating systems,

Combinations of hardware and software that work together.

System Test – Common defects and failures:

Functional failures: System cannot complete tasks correctly, including end-to-end workflows.

Environment failures: System does not work properly in the intended environment(s) (hardware, OS, network).

Documentation mismatches: System behavior differs from what's described in manuals or requirements. E.g: wrong calculation results.

e.g. incorrect calculation results (documentation mismatch), incorrect / unexpected (non-)functional system behavior, incorrect control and/or data flows within the system

System Test – Test Environment:

The test environment is the environment where the system will be tested.

For the system testing the test environment shall simulate the production environment or deployment environment where the system will run for the user.

It shall use the same hardware, databases, OS, network protocols, and so on..

Since an incomplete environment can hide some defects and failures

Should system tests be run in production? Why? Why not?:

NO, it shall not. Here is why. There are many disadvantages when testing in production:

If a failure happens it affect's the user's environment.

This failure can also cause data loss for the costumer or other expensive damage.

No / little control over configuration of the production environment; meaning potential changes to test conditions due to operation running parallel to the test; since we are testing in production the testing environment can receive data unpredictably since real users are using the system as you are testing, giving the tester almost no control over the test environment (production)

System tests are difficult to reproduce / no longer reproducible.

System Test – Test Strategy:;

Requirement-based testing: we use the requirements document as our test basis, at least on system test case per requirement, usually we have more than one requirement test base.

Business process-based testing: Create tests that simulate real business processes. Focus on important processes, people, systems, and context. Prioritize tests based on frequency and relevance of processes.

Use case-based testing: the tester separate users into user groups. Each with their own repetitive behavior. Then we build and make use cases based on these groups to test the system. priority of these actions (use cases) are based on how often does an action occur.

Non-functional Testing: these tests don't test the features but rather the reliability, security, robustness of the system.

Non-functional tests and their meaning:

Performance Test: it focuses on response time consistency when the load increase

Load Test: how the system handles low usage load, and peak (known) load.

Volume Testing: its not about how many people are using the system or how many requests are sent it about the volume of data

Test of (data) security: against unauthorized system access or data access

Reliability Test: checks how stable and dependable a system is over time. It measures continuous operation, seeing how long the system can run without failing.

Resilience and Robustness test: test how the system handles incorrect data, failure. Whether the system handles errors, failures and crashes with grace. And often called chaos testing.

Compatibility test / data conversion test: compatibility test tests whether the system integrates and works well with other system, e.g linux. Data conversion test focus on whether the system can import, export, handle data well between different system and/or formats.

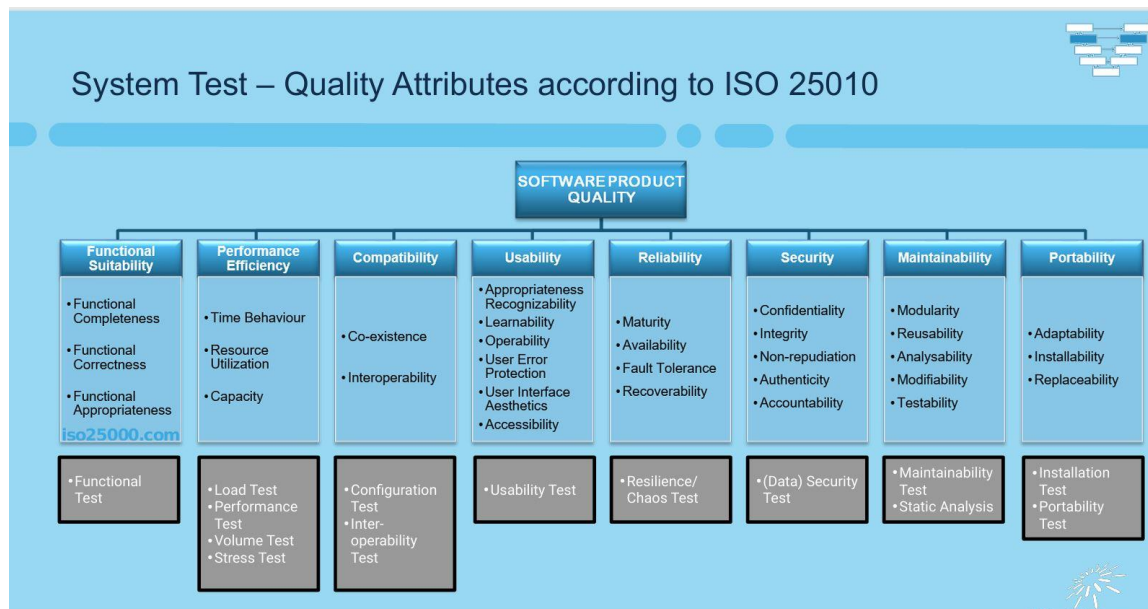
Configuration Testing: checks how the system behaves under different setups or configurations. For instance makes sure that the system works on different web browser.

Usability Testing: focuses on whether the user can use the system easily, whether the system is intuitive.

Checking documentation: makes sure the system behaves as described in manuals, guides, or instructions.

Check for changeability/maintainability: tests how easy it is to understand, modify, and maintain the system over time.

As a picture for functional and non-functional testing according to ISO:



Test Types Terms:

Functional quality attributes: Check if the system is complete, correct, and appropriate for its purpose. It also verifies functional requirements and what the system shall do.

Non-functional test attributes: Check if the system is reliable, fast, efficient, usable, secure, etc. non-functional tests check non-functional requirements

Correctness and completeness of component architecture: Verify that all parts of the system are correctly designed and integrated.

Effects of changes: Test whether modifications, updates, or fixes affect the system unexpectedly.

White-Box Tests / White-Box Methods: Test internal structures, code, and logic of the system.

Change driven tests: tests that you conduct whenever there has been a change in the system and we have 2 types of them

Confirmation Testing: verify that the defects that was before discovered are now fixed

Regression Testing: test that the components that were already tested before are still correct after the change.

Test Levels and test Types:

Any type of test can be applied at any level of test levels however the intensity of the test differs depends on the testing level.

Confirmation Test Vs Regression Test:

Confirmation testing: is performed to verify that a previously identified defect has been successfully fixed. If the defect involved a missing functionality, confirmation testing may also include new tests to validate that the functionality now exists and works correctly.

Regression Test: retesting software that has already been tested to ensure that recent changes or updates have not introduced new defects or affected existing functionality. Its primary goal is not to find new errors, but to confirm that previously working areas of the system continue to work correctly after modifications. Regression tests are important even if the software environment changes, because updates, patches, or configuration changes can inadvertently impact previously tested functionality. The scope of regression testing can vary—it may involve retesting only the parts affected by recent changes or a broader set of test cases to cover critical or high-risk areas of the system.

Perform confirmation tests and regression tests (repeatedly) at all test levels!

Scope of Regression Testing:

Full regression: test involves rerunning all existing test cases from the initial testing of the software to ensure that no functionality has been broken by recent changes. Each test case is treated with the same level of importance as when the software was first tested, providing full verification of the system. Complete regression testing is also important if the system environment has changed, such as updates to hardware, operating systems, or configurations, because these changes could introduce new issues. However, in practice, performing a complete regression test is often too expensive and time-consuming, so selective or partial regression testing is usually performed instead.

When a complete regression test is too expensive, testers select a subset of test cases.

Factors for the selection of test cases for the regression:

Repeat high priority tests only

Focus on specific configurations (e.g. language, OS)

Focus on software impacted by changes

Skip rare or unusual scenarios that are unlikely to be affected by the recent changes.

Regression test suites often executed with few changes. Excellent candidates for test automation

Maintenance Test:

The development ends when the system pass the acceptance testing. However, the testing never ends most system are in production for years and they get often updated, each update or environment change requires some testing.

Types of Maintenance testing:

Additive Maintenance: Adding new functionality or features that weren't in the original release.

Modification and extension work planned from the outset Normal product development
Software project not completed with delivery of the first version.

Perfective Maintenance: Improving performance, usability, or maintainability of existing features without changing functionality.

Improvement of quality factors, e.g. maintainability, performance, usability without changes in functional scope

Adaptive Maintenance: Making the system compatible with changes in the environment, like OS upgrades, hardware changes, or connected systems.

OS updates, DB management system updates, Upgrades of commercial software, Patches of external components.

Corrective Maintenance (troubleshooting): Fixing defects or errors discovered after the system is in use.

Errors that occurred operation Emergency fixes (“hot fixes”)

Which is which:

Planned ongoing development (additive)

Performance improvement (perfective)

Changing the environment (adaptive)

Troubleshooting (corrective)

System refactoring (perfective)

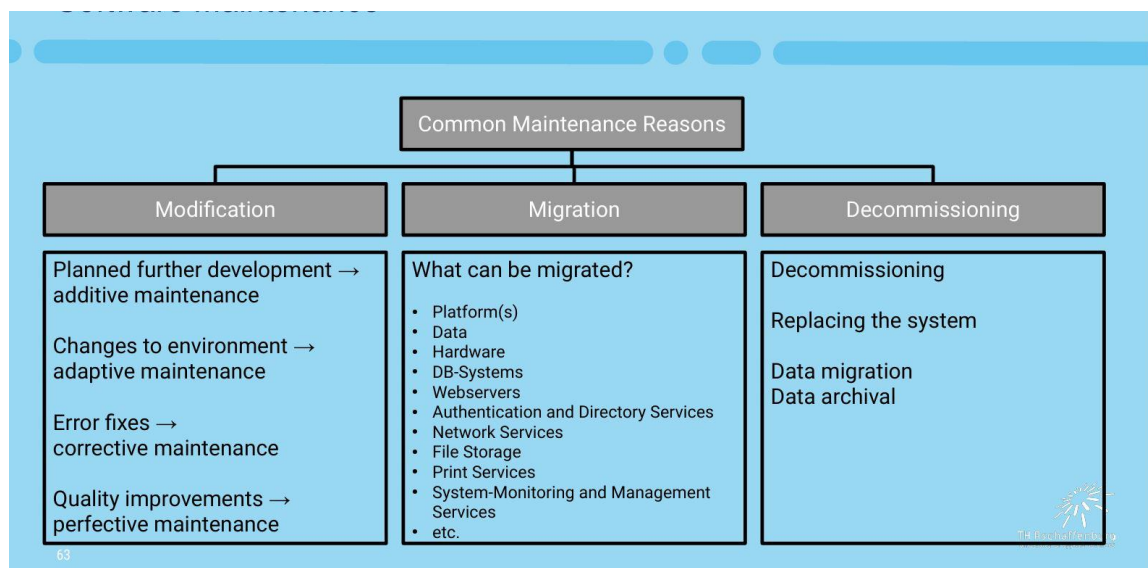
Changes due to other changes in a neighboring system (depends)

Implementation of a planned but unfinished functionality (additive)

Extensions for a planned market expansion (adaptive)

Reasons for Maintenance:

Decommissioning: removing a system, application, or component from active use.



Maintenance Test during Modification:

Maintenance during modification means performing tests and checks while the system is being changed or updated.

When doing system maintenance we might require tests on several testing level.

To plan maintenance tests effectively testers should do impact analysis

Deciding the scope of the tests depends:

Change risk level: How risky the change is. For example, a change affecting critical system communication or high-traffic modules requires more thorough testing.

Existing system size: Larger systems may need broader testing because changes can have more unexpected effects across the system.

Size of change: Larger systems may need broader testing because changes can have more unexpected effects across the system.

Maintenance Test during Migration/Decommissioning:

Maintenance test during migration: when the system is moved to a new environment for instance a new OS. The system shall be tested that it works correctly on the new environment.

Maintenance test during decommissioning:

Data migration: Make sure data is correctly moved to the new system, even if formats change.

Archiving: Verify long-term storage of data.

Recovery: Test that archived data can be restored if needed.

Regression testing: Check that any parts of the system still in use continue to work properly.

during migration or system replacement, you often need to convert data from one format or system to another. Conversion tests make sure this process works correctly.

We should test the converting software, and the data that is converted.